

Enhancing Medical Question-Answering with Qwen3-0.6B Using the MedQuAD Dataset

Project README Documentation

Project Overview

This project explores the fine-tuning and deployment of **Qwen3-0.6B**, a lightweight Large Language Model (LLM), for **medical question answering (QA)** using the **MedQuAD dataset**, which contains 16,000+ expert-verified QA pairs. The goal is to determine whether small LLMs can provide accurate, interpretable, and computationally efficient medical responses, even when deployed on CPUs using Docker.

Research Objectives

- Fine-tune Qwen3-0.6B using MedQuAD data.
- Evaluate answer quality using BLEU, ROUGE-L, F1-score, Exact-Match, METEOR, and Perplexity.
- Optimize deployment using LoRA, quantization, and CPU inference via Docker.
- Demonstrate lightweight inferences for healthcare AI support systems.

Keywords

Medical QA, Qwen3-0.6B, MedQuAD, Healthcare AI, Docker Deployment, LoRA, CPU Inference.

1 Project Workflow

1. **EDA** – Clean and explore MedQuAD dataset.
2. **Fine-Tuning** – LoRA-based tuning; output stored in `my-qwen-model`.
3. **Evaluation** – BLEU, ROUGE-L, F1, METEOR, Perplexity, Human Assessment.
4. **Inference** – Run via Jupyter, API, or Docker.

2 Model Advantages

- Lightweight LLM with strong accuracy.
- Optimized using LoRA and quantization.
- Deployable on CPU and Docker.
- Excellent for healthcare QA applications.

3 Evaluation Metrics

BLEU	Linguistic accuracy.
ROUGE-L	Conceptual overlap.
F1 Score	Precision-recall balance.
Exact-Match	Full answer equivalence.
METEOR	Synonym-aware similarity.
Perplexity	Language fluency.
Human Evaluation	Clinical correctness.

4 Deployment (Docker & CPU)

To ensure broad accessibility, we provide a lightweight Docker-based deployment process that works on standard consumer CPUs. We do not ship heavy base model weights inside the image; instead, we mount them via Docker volumes.

4.1 Prerequisites

- Docker Desktop installed and running.
- Fine-tuned LoRA adapter folder: `my-qwen-model/`

4.2 Project Structure

```
MyProject/  
  my-qwen-model/    # Custom LoRA adapter & Tokenizer  
  requirements.txt  # Python dependencies  
  deployment/       # Deployment logic  
    Dockerfile  
    main.py
```

4.3 Build Docker Image

Navigate to the project root and build the Docker image:

```
docker build -t med-chat-api -f deployment/Dockerfile .
```

4.4 Run the Container

```
docker run -p 8000:8000 \  
  -v ${PWD}/my-qwen-model:/app/my-qwen-model \  
  --name med-chat \  
  med-chat-api
```

Why this setup?

- `-v hf_cache`: Download base Qwen model only once.
- `-v my-qwen-model`: Swap adapters without rebuilding the Docker image.

4.5 Access the API

Once Docker logs show API Ready, test via:

Swagger Interface: <http://localhost:8000/docs>

Command-line Test:

```
curl -X 'POST' \
  'http://localhost:8000/generate' \
  -H 'Content-Type: application/json' \
  -d '{
    "prompt": "What are the symptoms of diabetes?",
    "max_new_tokens": 100,
    "temperature": 0.7
  }'
```

4.6 Performance Note

Inference latency will be higher on CPU. First-time execution may take several minutes as the base model downloads and LoRA adapters are loaded.

5 Future Enhancements

- Real-time medical chatbot integration.
- Multilingual response capabilities.
- Hallucination detection.
- Integration with PubMed and clinical knowledge sources.

6 License

This project is released under the **MIT License** for research and academic use.

7 Acknowledgements

We acknowledge Hugging Face Transformers, Qwen (Alibaba Cloud), the MedQuAD dataset, NIH, and MedlinePlus for enabling this project.